North Bengal International University

Lab Report: 02

Exp Name : Subtraction of two Binary number using 2's complement.

Subject : Computer Architecture & Organization Sessional

Subject Code : CSE 0612-3142

Submitted By:

Name : Md. Tanvir Rahman Pranto
Dept : CSE
Roll : 0712340012101005
Semester : 2nd Year, 4th semester.

Submitted To:

Name : Md. Shafiuzzaman
Designation : Lecturer
Dept : CSE
Date :

Experiment No: 02

Experiment Name: Subtraction of two Binary number using 2's complement.

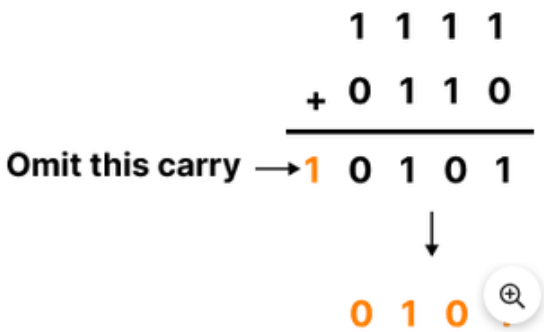
Theory: 2's complement is a popular method for performing binary subtraction because it allows subtraction to be handled as an addition operation. To subtract one binary number from another, you first convert the subtrahend into its 2's complement by inverting all bits (creating the 1's complement) and then adding 1. This transforms the subtrahend into its negative equivalent. Adding the 2's complement of the subtrahend to the minuend yields the desired difference, and any resulting overflow carry is discarded. This method not only simplifies the hardware design for arithmetic operations but also efficiently represents both positive and negative numbers within fixed-bit systems.

Process:

- **Write the Numbers in Binary:** Represent both numbers in binary with the same number of bits (for example, 5 bits). Let A be the minuend (the number from which you subtract) and B be the subtrahend (the number to subtract).
- **Calculate the 1's Complement of B:** Flip each bit of B. In other words, change every 0 to 1 and every 1 to 0.
- **Obtain the 2's Complement of B:** Add 1 to the 1's complement of B. This result is the 2's complement and represents the negative of B.
- **Add A and the 2's Complement of B:** Perform binary addition of A and the 2's complement of B.
    - Start adding from the least significant bit (rightmost bit) and move towards the most significant bit.
    - Keep track of any carry generated during the addition.
- **Handle the Overflow (if any):** In 2's complement arithmetic, if there is an extra carry out from the most significant bit, you simply ignore it.
- **Interpret the Result:** The resulting binary number represents the difference A − B. If the most significant bit is 1 (in a fixed bit-width system), it indicates a negative result.

Here shown the subtract $(1010)_2$ from $(1111)_2$ using 2's complement method.
Step1**:** 2's complement of (1010) is (0110)
Step2: Add (0110) to (1111)

This is shown below:

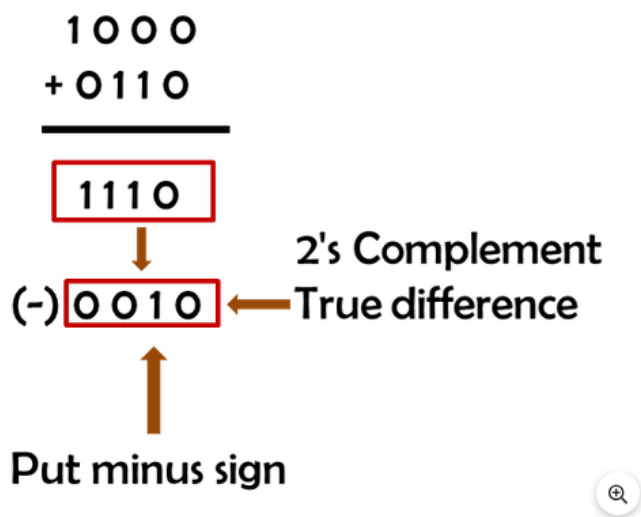


Subtract Using 2's Complement Method

Here shown the subtraction $(1010)_2$ from $(1000)_2$ using 1's complement method.

Step1**:** 2's complement of (1010) is (0110)

Step2: Add(0110) to (1000)



2's complement

Code:

```
 #include <stdio.h>

int main() {
    int A[5], B[5], oneCom[5], twoCom[5], Sum[6], carry = 1;

    // Input A
    printf("Enter 5-bit binary A: ");
    for (int i = 0; i < 5; i++) {
        scanf("%d", &A[i]);
    }

    // Input B
    printf("Enter 5-bit binary B: ");
    for (int i = 0; i < 5; i++) {
        scanf("%d", &B[i]);
        oneCom[i] = 1 - B[i]; // 1's complement of B
    }

    // Display 1's Complement of B
    printf("\n1's Complement of B: ");
    for (int i = 0; i < 5; i++) {
        printf("%d", oneCom[i]);
    }
    printf("\n");

    // Compute 2's complement of B by adding 1 to 1's complement
    for (int i = 4; i >= 0; i--) {
        int sumBit = oneCom[i] + carry;
        twoCom[i] = sumBit % 2;
        carry = sumBit / 2;
    }

    // Display 2's Complement of B
    printf("\n2's Complement of B: ");
    for (int i = 0; i < 5; i++) {
        printf("%d", twoCom[i]);
    }
    printf("\n");

    // Perform A + 2's complement of B
    carry = 0;
    for (int i = 4; i >= 0; i--) {
        int sumBit = A[i] + twoCom[i] + carry;
        Sum[i + 1] = sumBit % 2;
        carry = sumBit / 2;
    }
    Sum[0] = carry;

    // Print subtraction result (A - B)
```

```
    printf("\nResult (A - B): ");
    for (int i = 0; i < 6; i++) {
        printf("%d", Sum[i]);
    }
    printf("\n");

    return 0;
}
```

Output:

```
OUTPUT    TERMINAL

Enter 5-bit binary A: 1 0 0 1 1
Enter 5-bit binary B: 1 0 1 1 1

1's Complement of B: 01000

2's Complement of B: 01001

Result (A - B): 011100
PS C:\Users\Lenovo\Documents\4th semester\computer architecture>
```

Conclusion: Conclusion: The 2's complement method transforms the complexity of binary subtraction into the simplicity of binary addition. By reflecting on the bits—flipping and unifying them through a little extra—we find that even the harshest differences can be harmonized. This approach not only lightens the load on digital systems but also mirrors the human spirit: by rethinking and reshaping challenges, we often discover a more elegant path to clarity.